Tool Usage in Kani

Provide Plain Python Code to Models

- Tools are written as plain Python methods and decorated with the special `@ai_function` decorator
- In this example, we give a model access to a real-time weather API
- Kani guarantees that parameters are valid by the time they reach developer code

```
class WeatherKani(Kani):
  @ai_function()
  def get_weather(self, loc: Annotated[str,
    AIParam(desc="The desired city")]):
    """Get the weather in a given location."""
    # ... Query some weather API
    return weather

engine = OpenAIEngine(api_key, model="gpt-4")
chat_in_terminal(WeatherKani(engine))
# USER: What's the weather in San Francisco?
# AI: [get_weather({"loc": "San Francisco"})]
# TOOL: {"temp": 72, ...}
# AI: It's currently 72F in San Francisco.
```

Tool Usage in Kani

Provide Plain Python Code to Models

- Tools are written as plain Python methods and decorated with the special `@ai_function` decorator
- In this example, we give a model access to a real-time weather API
- Kani guarantees that parameters are valid by the time they reach developer code

```
class WeatherKani(Kani):
  @ai_function()
  def get_weather(self, loc: Annotated[str,
    AIParam(desc="The desired city")]):
    """Get the weather in a given location."""
    # ... Query some weather API
    return weather

engine = OpenAIEngine(api_key, model="gpt-4")
chat_in_terminal(WeatherKani(engine))
# USER: What's the weather in San Francisco?
# AI: [get_weather({"loc": "San Francisco"})]
# TOOL: {"temp": 72, ...}
# AI: It's currently 72F in San Francisco.
```